Cafeteria Management System

Database-Heavy University Semester Project (Oracle 19c + Flask)

Generated: 2026-05-08 21:26

1. System Overview

A cafeteria serving students, staff, and guests. The system supports menu management, ordering, inventory tracking via ingredient usage, employee shifts, billing/payments, and customer feedback.

2. Database Schema (Normalized to 3NF/BCNF)

Core entities include Customer, Employee, MenuCategory, MenuItem, Order, OrderItem, Inventory, IngredientUsage, Payment, Feedback, Discount, and Shift. The design separates repeating groups (OrderItem) and many-to-many relationships (IngredientUsage) and uses constraints/triggers to enforce business rules.

- **Order** → **OrderItem**: one-to-many line items.
- **MenuItem** → **IngredientUsage** → **Inventory**: bill-of-materials for stock deduction.
- **Order** → **Payment**: 1:1 logical mapping enforced by UNIQUE(Order_ID).
- **Order** → **Feedback**: 0/1 feedback per order enforced by UNIQUE(Order_ID).
- **Employee** → **Shift**: day-wise shifts, unique per employee per date.

3. DDL (Tables, Constraints, Sequences)

The DDL creates all tables with primary keys, foreign keys, uniqueness, and CHECK constraints. The table name "ORDER" is quoted (Oracle keyword).

```
/* ========================================================================
   Cafeteria Management System (Oracle 19c)
   File: 01_ddl.sql
   Notes:
     - IMPORTANT: run as a SCRIPT (SQL Developer: F5), not statement-by-statement,
       because the cleanup uses PL/SQL blocks terminated with '/'.
     - "ORDER" is a quoted table name and must be referenced as "ORDER".
   ======================================================================== */

/* ------------------------------------------------------------------------
   OPTIONAL CLEANUP (safe drops)
   ------------------------------------------------------------------------ */
BEGIN EXECUTE IMMEDIATE 'DROP VIEW LowInventory'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW PopularItems'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP VIEW DailyRevenue'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP TYPE OrderSummaryType FORCE'; EXCEPTION WHEN OTHERS THEN NULL; E
/

BEGIN EXECUTE IMMEDIATE 'DROP TABLE SHIFT CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN NUL
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE PAYMENT CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN N
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE FEEDBACK CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE INGREDIENT_USAGE CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHE
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE INVENTORY CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE ORDER_ITEM CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THE
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE "ORDER" CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN N
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE DISCOUNT CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE MENU_ITEM CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE MENU_CATEGORY CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE EMPLOYEE CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
/
BEGIN EXECUTE IMMEDIATE 'DROP TABLE CUSTOMER CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN
/

BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_CUSTOMER'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_EMPLOYEE'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_MENU_CATEGORY'; EXCEPTION WHEN OTHERS THEN NULL; EN
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_MENU_ITEM'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_ORDER'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_ORDER_ITEM'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_INVENTORY'; EXCEPTION WHEN OTHERS THEN NULL; END;
```

```
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_INGREDIENT_USAGE'; EXCEPTION WHEN OTHERS THEN NULL;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_FEEDBACK'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_PAYMENT'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_DISCOUNT'; EXCEPTION WHEN OTHERS THEN NULL; END;
/
BEGIN EXECUTE IMMEDIATE 'DROP SEQUENCE SEQ_SHIFT'; EXCEPTION WHEN OTHERS THEN NULL; END;
/

/* ------------------------------------------------------------------------
   TABLE: CUSTOMER
   ------------------------------------------------------------------------ */
CREATE TABLE CUSTOMER (
  CUSTOMER_ID      NUMBER         NOT NULL,
  NAME             VARCHAR2(100)   NOT NULL,
  EMAIL            VARCHAR2(255)   NOT NULL,
  PHONE            VARCHAR2(20)    NOT NULL,
  CUSTOMER_TYPE    VARCHAR2(20)    NOT NULL,
  REGISTERED_DATE  DATE           DEFAULT TRUNC(SYSDATE) NOT NULL,
  CONSTRAINT PK_CUSTOMER PRIMARY KEY (CUSTOMER_ID),
  CONSTRAINT UQ_CUSTOMER_EMAIL UNIQUE (EMAIL),
  CONSTRAINT UQ_CUSTOMER_PHONE UNIQUE (PHONE),
  CONSTRAINT CK_CUSTOMER_TYPE CHECK (CUSTOMER_TYPE IN ('Student','Staff','Guest')),
  -- Oracle REGEXP_LIKE uses POSIX ERE; keep patterns Oracle-compatible.
  CONSTRAINT CK_CUSTOMER_EMAIL CHECK (REGEXP_LIKE(EMAIL, '^[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\
  CONSTRAINT CK_CUSTOMER_PHONE CHECK (REGEXP_LIKE(PHONE, '^[0-9+ \-]{7,20}$'))
);

/* ------------------------------------------------------------------------
   TABLE: EMPLOYEE
   - ShiftStart/ShiftEnd represent a default daily shift window; actual shifts
     are recorded in SHIFT table.
   ------------------------------------------------------------------------ */
CREATE TABLE EMPLOYEE (
  EMPLOYEE_ID   NUMBER         NOT NULL,
  NAME          VARCHAR2(100)  NOT NULL,
  ROLE          VARCHAR2(20)   NOT NULL,
  SALARY        NUMBER(10,2)   NOT NULL,
  HIRE_DATE     DATE           NOT NULL,
  SHIFT_START   VARCHAR2(5)    NOT NULL,
  SHIFT_END     VARCHAR2(5)    NOT NULL,
  CONSTRAINT PK_EMPLOYEE PRIMARY KEY (EMPLOYEE_ID),
  CONSTRAINT CK_EMPLOYEE_ROLE CHECK (ROLE IN ('Cashier','Chef','Manager')),
  CONSTRAINT CK_EMPLOYEE_SALARY CHECK (SALARY >= 0),
  CONSTRAINT CK_EMPLOYEE_SHIFTSTART CHECK (REGEXP_LIKE(SHIFT_START, '^([01][0-9]|2[0-3]):[0-5]
  CONSTRAINT CK_EMPLOYEE_SHIFTEND CHECK (REGEXP_LIKE(SHIFT_END, '^([01][0-9]|2[0-3]):[0-5][0-9
);

/* ------------------------------------------------------------------------
   TABLE: MENU_CATEGORY
   ------------------------------------------------------------------------ */
CREATE TABLE MENU_CATEGORY (
  CATEGORY_ID    NUMBER         NOT NULL,
  CATEGORY_NAME  VARCHAR2(30)   NOT NULL,
  DESCRIPTION    VARCHAR2(400),
  CONSTRAINT PK_MENU_CATEGORY PRIMARY KEY (CATEGORY_ID),
  CONSTRAINT UQ_MENU_CATEGORY_NAME UNIQUE (CATEGORY_NAME),
  CONSTRAINT CK_MENU_CATEGORY_NAME CHECK (CATEGORY_NAME IN ('Main','Beverage','Snack','Dessert
);

/* ------------------------------------------------------------------------
   TABLE: MENU_ITEM
```

```
   - AverageRating is denormalized and maintained via trigger after FEEDBACK.
   ------------------------------------------------------------------------------ */
CREATE TABLE MENU_ITEM (
  ITEM_ID         NUMBER         NOT NULL,
  CATEGORY_ID     NUMBER         NOT NULL,
  NAME            VARCHAR2(120)  NOT NULL,
  PRICE           NUMBER(8,2)    NOT NULL,
  CALORIE_COUNT   NUMBER(6)      NOT NULL,
  IS_AVAILABLE    CHAR(1)        DEFAULT 'Y' NOT NULL,
  AVERAGE_RATING  NUMBER(3,2)    DEFAULT 0   NOT NULL,
  CONSTRAINT PK_MENU_ITEM PRIMARY KEY (ITEM_ID),
  CONSTRAINT FK_MENU_ITEM_CATEGORY FOREIGN KEY (CATEGORY_ID) REFERENCES MENU_CATEGORY(CATEGORY
  CONSTRAINT UQ_MENU_ITEM_NAME UNIQUE (NAME),
  CONSTRAINT CK_MENU_ITEM_PRICE CHECK (PRICE >= 0),
  CONSTRAINT CK_MENU_ITEM_CAL CHECK (CALORIE_COUNT >= 0),
  CONSTRAINT CK_MENU_ITEM_AVAIL CHECK (IS_AVAILABLE IN ('Y','N')),

... (truncated) ...
```

4. DCL (Roles & Privileges)

Two roles are created: cafeteria_viewer (read-only on selected tables) and cafeteria_cashier (insert/update on operational tables).

```
/* ================================================================================
   Cafeteria Management System (Oracle 19c)
   File: 02_dcl.sql
   Purpose:
     - Create roles required by the application
     - Grant/revoke privileges exactly as requested

   Notes:
     - The table "ORDER" is quoted and must be referenced as "ORDER".
   ================================================================================ */

/* --------------------------------------------------------------------------------
   ROLES
   -------------------------------------------------------------------------------- */
BEGIN
  EXECUTE IMMEDIATE 'CREATE ROLE cafeteria_viewer';
EXCEPTION
  WHEN OTHERS THEN
    IF SQLCODE != -1921 THEN RAISE; END IF; -- ORA-01921: role name already exists
END;
/

BEGIN
  EXECUTE IMMEDIATE 'CREATE ROLE cafeteria_cashier';
EXCEPTION
  WHEN OTHERS THEN
    IF SQLCODE != -1921 THEN RAISE; END IF;
END;
/

/* --------------------------------------------------------------------------------
   GRANTS
   -------------------------------------------------------------------------------- */
-- Viewer role: read-only access to key operational tables
GRANT SELECT ON "ORDER"   TO cafeteria_viewer;
GRANT SELECT ON MENU_ITEM TO cafeteria_viewer;
GRANT SELECT ON FEEDBACK  TO cafeteria_viewer;

-- Cashier role: allowed to record orders, order items, and payments
GRANT INSERT, UPDATE ON "ORDER"     TO cafeteria_cashier;
GRANT INSERT, UPDATE ON ORDER_ITEM  TO cafeteria_cashier;
GRANT INSERT, UPDATE ON PAYMENT     TO cafeteria_cashier;

/* --------------------------------------------------------------------------------
   REVOKES
   -------------------------------------------------------------------------------- */
-- Explicitly revoke DROP (requested). Note: roles do not receive DROP on objects
-- unless granted system privileges; this ensures it is not present.
BEGIN
  EXECUTE IMMEDIATE 'REVOKE DROP ANY TABLE FROM cafeteria_cashier';
EXCEPTION
  WHEN OTHERS THEN
    -- If the role never had the privilege, Oracle raises an error; ignore it.
    -- (Common message: "system privileges not granted to 'CAFETERIA_CASHIER'")
    IF SQLCODE IN (-1952, -1927, -1031) THEN
      NULL;
    ELSE
      RAISE;
    END IF;
```

```
END;
/

/* ------------------------------------------------------------------------
   END
   ------------------------------------------------------------------------ */
```

5. Data Population (Mock Data)

Seed data includes realistic Pakistani/university names, cafeteria menu items with prices/calories, inventory ingredients, ingredient usage mappings, orders, payments, and feedback.

```
/* ============================================================================
   Cafeteria Management System (Oracle 19c)
   File: 03_data.sql
   Purpose:
     - Populate all tables with realistic mock data (10–15 rows each)
     - Data is consistent with constraints and supports queries/PLSQL demos

   Notes:
     - Uses explicit IDs for easy referencing in queries/PLSQL.
     - "ORDER" is quoted and must be referenced as "ORDER".
   ============================================================================ */

/* ----------------------------------------------------------------------------
   MENU_CATEGORY (4)
   ---------------------------------------------------------------------------- */
INSERT INTO MENU_CATEGORY (CATEGORY_ID, CATEGORY_NAME, DESCRIPTION) VALUES (1,'Main','Main mea
INSERT INTO MENU_CATEGORY (CATEGORY_ID, CATEGORY_NAME, DESCRIPTION) VALUES (2,'Beverage','Hot 
INSERT INTO MENU_CATEGORY (CATEGORY_ID, CATEGORY_NAME, DESCRIPTION) VALUES (3,'Snack','Quick b
INSERT INTO MENU_CATEGORY (CATEGORY_ID, CATEGORY_NAME, DESCRIPTION) VALUES (4,'Dessert','Sweet

/* ----------------------------------------------------------------------------
   CUSTOMER (12)
   ---------------------------------------------------------------------------- */
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES

INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES

INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES
INSERT INTO CUSTOMER (CUSTOMER_ID, NAME, EMAIL, PHONE, CUSTOMER_TYPE, REGISTERED_DATE) VALUES

/* ----------------------------------------------------------------------------
   EMPLOYEE (10)
   ---------------------------------------------------------------------------- */
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU
INSERT INTO EMPLOYEE (EMPLOYEE_ID, NAME, ROLE, SALARY, HIRE_DATE, SHIFT_START, SHIFT_END) VALU

/* ----------------------------------------------------------------------------
   DISCOUNT (3) - valid for current semester window
   ---------------------------------------------------------------------------- */
INSERT INTO DISCOUNT (DISCOUNT_ID, CUSTOMER_TYPE, DISCOUNT_PERCENT, VALID_FROM, VALID_TO)
VALUES (1,'Student',10,DATE '2026-01-01',DATE '2026-12-31');
INSERT INTO DISCOUNT (DISCOUNT_ID, CUSTOMER_TYPE, DISCOUNT_PERCENT, VALID_FROM, VALID_TO)
VALUES (2,'Staff',5,DATE '2026-01-01',DATE '2026-12-31');
```

```
INSERT INTO DISCOUNT (DISCOUNT_ID, CUSTOMER_TYPE, DISCOUNT_PERCENT, VALID_FROM, VALID_TO)
VALUES (3,'Guest',0,DATE '2026-01-01',DATE '2026-12-31');

/* ------------------------------------------------------------------------
   MENU_ITEM (15)
   ------------------------------------------------------------------------ */
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (1,1,'Chicken Biryani Plate',250,780,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (2,1,'Beef Burger',320,650,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (3,1,'Zinger Burger',350,720,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (4,1,'Chicken Karahi (Single)',450,900,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (5,1,'Daal Chawal',180,620,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (6,2,'Masala Chai',80,120,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (7,2,'Cold Coffee',220,260,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (8,2,'Fresh Lime Soda',150,110,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (9,3,'Samosa (2 pcs)',90,300,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (10,3,'French Fries',160,420,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (11,3,'Chicken Patty',120,360,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (12,4,'Gulab Jamun (2 pcs)',140,330,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (13,4,'Kheer Cup',130,290,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (14,4,'Chocolate Brownie',200,410,'Y',0);
INSERT INTO MENU_ITEM (ITEM_ID, CATEGORY_ID, NAME, PRICE, CALORIE_COUNT, IS_AVAILABLE, AVERAGE
VALUES (15,2,'Mineral Water (500ml)',60,0,'Y',0);

/* ------------------------------------------------------------------------
   INVENTORY (15 ingredients)
   ------------------------------------------------------------------------ */
INSERT INTO INVENTORY (INVENTORY_ID, ITEM_NAME, UNIT, QUANTITY_IN_STOCK, REORDER_LEVEL, LAST_R
VALUES (1,'Basmati Rice','kg',80,20,DATE '2026-05-01');
INSERT INTO INVENTORY VALUES (2,'Chicken Boneless','kg',55,15,DATE '2026-05-02');
INSERT INTO INVENTORY VALUES (3,'Beef Mince','kg',25,10,DATE '2026-05-03');
INSERT INTO INVENTORY VALUES (4,'Burger Buns','pcs',200,60,DATE '2026-05-02');
INSERT INTO INVENTORY VALUES (5,'Cooking Oil','l',60,15,DATE '2026-04-28');
INSERT INTO INVENTORY VALUES (6,'Tea Leaves','g',4000,1200,DATE '2026-05-02');
INSERT INTO INVENTORY VALUES (7,'Milk','l',90,25,DATE '2026-05-02');
INSERT INTO INVENTORY VALUES (8,'Sugar','kg',35,10,DATE '2026-04-25');
INSERT INTO INVENTORY VALUES (9,'Potatoes','kg',70,20,DATE '2026-05-01');
INSERT INTO INVENTORY VALUES (10,'Flour (Maida)','kg',50,15,DATE '2026-04-30');
INSERT INTO INVENTORY VALUES (11,'Lentils (Daal Mix)','kg',40,12,DATE '2026-04-29');
INSERT INTO INVENTORY VALUES (12,'Spice Mix (Garam Masala)','g',2500,800,DATE '2026-05-01');
INSERT INTO INVENTORY VALUES (13,'Lemons','pcs',180,60,DATE '2026-05-04');
INSERT INTO INVENTORY VALUES (14,'Cocoa Powder','g',1200,400,DATE '2026-04-20');
INSERT INTO INVENTORY VALUES (15,'Mineral Water Bottles 500ml','pcs',350,120,DATE '2026-05-03'

/* ------------------------------------------------------------------------
   INGREDIENT_USAGE (15 menu items, multiple ingredients each)
   QuantityRequired uses inventory units (e.g., kg, l, pcs, g).
   ------------------------------------------------------------------------ */

... (truncated) ...
```

6. Advanced SQL (Queries, Views, Indexes)

This script contains the required join queries, set operations, subqueries, views (DailyRevenue, PopularItems, LowInventory), and performance indexes.

```
/* ================================================================================
   Cafeteria Management System (Oracle 19c)
   File: 04_queries.sql
   Purpose:
     - Advanced SQL queries requested in the project
     - Joins, Set operations, Subqueries, Views, Indexes

   Notes:
     - "ORDER" is quoted and must be referenced as "ORDER".
   ================================================================================ */

/* --------------------------------------------------------------------------------
   JOINS
   -------------------------------------------------------------------------------- */

/* 1) INNER JOIN — all completed orders with customer name and total amount */
SELECT
  o.ORDER_ID,
  c.NAME AS CUSTOMER_NAME,
  o.ORDER_DATE,
  o.TOTAL_AMOUNT
FROM "ORDER" o
JOIN CUSTOMER c
  ON c.CUSTOMER_ID = o.CUSTOMER_ID
WHERE o.STATUS = 'Completed'
ORDER BY o.ORDER_DATE DESC, o.ORDER_ID DESC;

/* 2) LEFT OUTER JOIN — all menu items with their feedback (including no feedback)
      Feedback is per order; we derive per-item feedback by linking orders that
      contained the item.
*/
SELECT
  mi.ITEM_ID,
  mi.NAME AS ITEM_NAME,
  f.FEEDBACK_ID,
  f.RATING,
  f.COMMENTS,
  f.FEEDBACK_DATE
FROM MENU_ITEM mi
LEFT JOIN ORDER_ITEM oi
  ON oi.ITEM_ID = mi.ITEM_ID
LEFT JOIN FEEDBACK f
  ON f.ORDER_ID = oi.ORDER_ID
ORDER BY mi.ITEM_ID, f.FEEDBACK_DATE;

/* 3) FULL JOIN — all employees and their assigned orders (including unassigned)
      Unassigned orders are those with EMPLOYEE_ID IS NULL.
*/
SELECT
  e.EMPLOYEE_ID,
  e.NAME AS EMPLOYEE_NAME,
  o.ORDER_ID,
  o.STATUS,
  o.ORDER_DATE,
  o.TOTAL_AMOUNT
FROM EMPLOYEE e
FULL OUTER JOIN "ORDER" o
  ON o.EMPLOYEE_ID = e.EMPLOYEE_ID
ORDER BY NVL(e.EMPLOYEE_ID, 999999), NVL(o.ORDER_ID, 999999);
```

```
/* ------------------------------------------------------------------------
   SET OPERATIONS
   ------------------------------------------------------------------------ */

/* 4) UNION — combine customer names from Student and Staff types */
SELECT NAME FROM CUSTOMER WHERE CUSTOMER_TYPE = 'Student'
UNION
SELECT NAME FROM CUSTOMER WHERE CUSTOMER_TYPE = 'Staff'
ORDER BY NAME;

/* 5) INTERSECT — find items that appear in both orders and inventory
      This is a business-specific demo: some menu items map directly to an
      inventory item by name (e.g., Mineral Water (500ml)).
*/
SELECT mi.NAME AS ITEM_NAME
FROM MENU_ITEM mi
INTERSECT
SELECT i.ITEM_NAME
FROM INVENTORY i;

/* 6) MINUS — find menu items never ordered */
SELECT mi.ITEM_ID, mi.NAME
FROM MENU_ITEM mi
MINUS
SELECT mi2.ITEM_ID, mi2.NAME
FROM MENU_ITEM mi2
JOIN ORDER_ITEM oi
  ON oi.ITEM_ID = mi2.ITEM_ID
ORDER BY ITEM_ID;

/* ------------------------------------------------------------------------
   SUBQUERIES
   ------------------------------------------------------------------------ */

/* 7) CORRELATED SUBQUERY — customers who spent above the average order amount */
SELECT
  c.CUSTOMER_ID,
  c.NAME,
  ROUND(SUM(o.TOTAL_AMOUNT), 2) AS LIFETIME_SPEND
FROM CUSTOMER c
JOIN "ORDER" o
  ON o.CUSTOMER_ID = c.CUSTOMER_ID
WHERE o.STATUS = 'Completed'
GROUP BY c.CUSTOMER_ID, c.NAME
HAVING SUM(o.TOTAL_AMOUNT) >
  (SELECT AVG(o2.TOTAL_AMOUNT)
   FROM "ORDER" o2
   WHERE o2.STATUS = 'Completed');

/* 8) NON-CORRELATED SUBQUERY — top 3 most ordered menu items (by quantity) */
SELECT *
FROM (
  SELECT
    mi.ITEM_ID,
    mi.NAME,
    SUM(oi.QUANTITY) AS TOTAL_QTY
  FROM MENU_ITEM mi
  JOIN ORDER_ITEM oi
    ON oi.ITEM_ID = mi.ITEM_ID
  GROUP BY mi.ITEM_ID, mi.NAME
  ORDER BY TOTAL_QTY DESC
)
WHERE ROWNUM <= 3;

/* ------------------------------------------------------------------------
```

```
   VIEWS
   ------------------------------------------------------------------------- */

/* 9) DailyRevenue — total revenue grouped by date
      Revenue uses Paid payments only.
*/
CREATE OR REPLACE VIEW DailyRevenue AS
SELECT
  TRUNC(p.PAYMENT_DATE) AS REVENUE_DATE,
  ROUND(SUM(p.AMOUNT_PAID), 2) AS TOTAL_REVENUE
FROM PAYMENT p
WHERE p.PAYMENT_STATUS = 'Paid'
GROUP BY TRUNC(p.PAYMENT_DATE);

/* 10) PopularItems — items ordered more than 5 times with avg rating
       Avg rating derived from feedback of orders that included the item.
*/
CREATE OR REPLACE VIEW PopularItems AS
SELECT
  mi.ITEM_ID,
  mi.NAME AS ITEM_NAME,
  SUM(oi.QUANTITY) AS TOTAL_ORDERED_QTY,
  ROUND(AVG(f.RATING), 2) AS AVG_RATING_FROM_FEEDBACK
FROM MENU_ITEM mi
JOIN ORDER_ITEM oi
  ON oi.ITEM_ID = mi.ITEM_ID
LEFT JOIN FEEDBACK f
  ON f.ORDER_ID = oi.ORDER_ID
GROUP BY mi.ITEM_ID, mi.NAME
HAVING SUM(oi.QUANTITY) > 5;

/* 11) LowInventory — all ingredients below reorder level */
CREATE OR REPLACE VIEW LowInventory AS
SELECT
  INVENTORY_ID,

... (truncated) ...
```

7. PL/SQL (Business Logic, Triggers, Object Types)

The CafeteriaManager package supports placing orders (parsing item tokens), applying discounts, creating payment records, and processing payments. Triggers maintain inventory deduction and denormalized average ratings.

```
/* ================================================================================
   Cafeteria Management System (Oracle 19c)
   File: 05_plsql.sql
   Purpose:
     - PL/SQL package (spec + body): CafeteriaManager
     - Cursors for pending orders and low-stock items
     - Triggers for IDs, inventory deduction, rating maintenance, delete protection
     - Object types for order summaries

   Notes:
     - "ORDER" is quoted and must be referenced as "ORDER".
     - PlaceOrder uses SYS.ODCIVARCHAR2LIST where each element is formatted as:
         '<ItemID>:<Quantity>'
       Example: SYS.ODCIVARCHAR2LIST('1:2','6:1','15:1')
   ================================================================================ */

/* --------------------------------------------------------------------------------
   OBJECT TYPE: OrderSummaryType
   -------------------------------------------------------------------------------- */
CREATE OR REPLACE TYPE OrderSummaryType AS OBJECT (
  OrderID       NUMBER,
  CustomerName  VARCHAR2(100),
  TotalAmount   NUMBER,
  Status        VARCHAR2(20),
  MEMBER FUNCTION GetLabel RETURN VARCHAR2
);
/

CREATE OR REPLACE TYPE BODY OrderSummaryType AS
  MEMBER FUNCTION GetLabel RETURN VARCHAR2 IS
  BEGIN
    RETURN 'Order #' || OrderID || ' - ' || CustomerName || ' (' || Status || ')';
  END;
END;
/

/* --------------------------------------------------------------------------------
   DEMO QUERY USING OBJECT TYPE
   -------------------------------------------------------------------------------- */
-- Returns a "table" (result set) of OrderSummaryType objects
SELECT
  OrderSummaryType(o.ORDER_ID, c.NAME, o.TOTAL_AMOUNT, o.STATUS) AS ORDER_SUMMARY
FROM "ORDER" o
JOIN CUSTOMER c ON c.CUSTOMER_ID = o.CUSTOMER_ID
ORDER BY o.ORDER_ID;

/* --------------------------------------------------------------------------------
   PACKAGE: CafeteriaManager (SPEC)
   -------------------------------------------------------------------------------- */
CREATE OR REPLACE PACKAGE CafeteriaManager AS
  /* Inserts a new order and its items, calculates totals with applicable discount.
     p_Items element format: '<ItemID>:<Quantity>' */
  PROCEDURE PlaceOrder(
    p_CustomerID IN NUMBER,
    p_EmployeeID IN NUMBER,
    p_Items      IN SYS.ODCIVARCHAR2LIST
  );
  /* Records or updates payment and updates order status accordingly */
```

```
  PROCEDURE ProcessPayment(
    p_OrderID IN NUMBER,
    p_Amount  IN NUMBER,
    p_Method  IN VARCHAR2
  );

  /* Returns lifetime completed spending for a customer */
  FUNCTION GetCustomerTotal(p_CustomerID IN NUMBER) RETURN NUMBER;

  /* Returns the average feedback rating for an item */
  FUNCTION GetAverageRating(p_ItemID IN NUMBER) RETURN NUMBER;
END CafeteriaManager;
/

/* ------------------------------------------------------------------------------
   PACKAGE: CafeteriaManager (BODY)
   ------------------------------------------------------------------------------ */
CREATE OR REPLACE PACKAGE BODY CafeteriaManager AS

  FUNCTION parse_item_id(p_token VARCHAR2) RETURN NUMBER IS
  BEGIN
    RETURN TO_NUMBER(REGEXP_SUBSTR(p_token, '^\s*([0-9]+)\s*:', 1, 1, NULL, 1));
  EXCEPTION
    WHEN OTHERS THEN
      RAISE_APPLICATION_ERROR(-20001, 'Invalid item token "'||p_token||'". Expected "<ItemID>:
  END;

  FUNCTION parse_qty(p_token VARCHAR2) RETURN NUMBER IS
  BEGIN
    RETURN TO_NUMBER(REGEXP_SUBSTR(p_token, ':\s*([0-9]+)\s*$', 1, 1, NULL, 1));
  EXCEPTION
    WHEN OTHERS THEN
      RAISE_APPLICATION_ERROR(-20002, 'Invalid quantity token "'||p_token||'". Expected "<Item
  END;

  FUNCTION get_discount_percent(p_customer_id NUMBER, p_order_date DATE) RETURN NUMBER IS
    v_type CUSTOMER.CUSTOMER_TYPE%TYPE;
    v_pct  DISCOUNT.DISCOUNT_PERCENT%TYPE;
  BEGIN
    SELECT CUSTOMER_TYPE INTO v_type FROM CUSTOMER WHERE CUSTOMER_ID = p_customer_id;

    SELECT NVL(MAX(DISCOUNT_PERCENT),0)
      INTO v_pct
      FROM DISCOUNT
     WHERE CUSTOMER_TYPE = v_type
       AND TRUNC(p_order_date) BETWEEN VALID_FROM AND VALID_TO;

    RETURN NVL(v_pct,0);
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RETURN 0;
  END;

  PROCEDURE PlaceOrder(
    p_CustomerID IN NUMBER,
    p_EmployeeID IN NUMBER,
    p_Items      IN SYS.ODCIVARCHAR2LIST
  ) IS
    v_order_id     NUMBER;
    v_total        NUMBER := 0;
    v_pct          NUMBER := 0;
    v_order_date   DATE := TRUNC(SYSDATE);
    v_order_time   VARCHAR2(5) := TO_CHAR(SYSDATE,'HH24:MI');
    v_item_id      NUMBER;
    v_qty          NUMBER;
```

```
    v_unit_price   NUMBER;
    v_item_avail   CHAR(1);
  BEGIN
    IF p_Items IS NULL OR p_Items.COUNT = 0 THEN
      RAISE_APPLICATION_ERROR(-20003, 'Order must contain at least one item.');
    END IF;

    v_order_id := SEQ_ORDER.NEXTVAL;

    INSERT INTO "ORDER"(ORDER_ID, CUSTOMER_ID, EMPLOYEE_ID, ORDER_DATE, ORDER_TIME, TOTAL_AMOU
    VALUES (v_order_id, p_CustomerID, p_EmployeeID, v_order_date, v_order_time, 0, 'Pending',

    FOR i IN 1 .. p_Items.COUNT LOOP
      v_item_id := parse_item_id(p_Items(i));
      v_qty     := parse_qty(p_Items(i));

      IF v_qty <= 0 THEN
        RAISE_APPLICATION_ERROR(-20004, 'Quantity must be > 0 for token "'||p_Items(i)||'".');
      END IF;

      SELECT PRICE, IS_AVAILABLE
        INTO v_unit_price, v_item_avail
        FROM MENU_ITEM
       WHERE ITEM_ID = v_item_id;

      IF v_item_avail <> 'Y' THEN
        RAISE_APPLICATION_ERROR(-20005, 'Menu item '||v_item_id||' is not available.');
      END IF;

      INSERT INTO ORDER_ITEM(ORDER_ITEM_ID, ORDER_ID, ITEM_ID, QUANTITY, UNIT_PRICE, SUBTOTAL)
      VALUES (SEQ_ORDER_ITEM.NEXTVAL, v_order_id, v_item_id, v_qty, v_unit_price, ROUND(v_qty

      v_total := v_total + ROUND(v_qty * v_unit_price, 2);
    END LOOP;

    v_pct := get_discount_percent(p_CustomerID, v_order_date);

... (truncated) ...
```

8. Flask Front-End (SQL/PLSQL only, no ORM)

The Flask app uses a single shared Oracle connection pool (python-oracledb Thin mode). All operations use SQL or call PL/SQL procedures.

8.1 Connection Pool (`app/db.py`)

```
import oracledb

def create_pool() -> oracledb.ConnectionPool:
    return oracledb.create_pool(
        user="cafeteria_admin",
        password="cafe1234",
        dsn="localhost:1521/XEPDB1",
        min=1,
        max=4,
        increment=1,
        timeout=60,
        wait_timeout=30,
        ping_interval=60,
    )

POOL = create_pool()

def fetchall(sql: str, params: dict | None = None) -> list[dict]:
    with POOL.acquire() as conn:
        with conn.cursor() as cur:
            cur.execute(sql, params or {})
            cols = [d[0].lower() for d in cur.description]
            return [dict(zip(cols, row)) for row in cur.fetchall()]

def execute(sql: str, params: dict | None = None) -> None:
    with POOL.acquire() as conn:
        with conn.cursor() as cur:
            cur.execute(sql, params or {})
        conn.commit()
```

8.2 Routes (`app/app.py`)

```
from __future__ import annotations

import datetime as dt

from flask import Flask, flash, redirect, render_template, request, url_for
import oracledb

from db import POOL


app = Flask(__name__)
app.secret_key = "cafeteria-secret-key-change-me"


def _as_date(value: str) -> dt.date:
    return dt.datetime.strptime(value, "%Y-%m-%d").date()


@app.get("/")
def root():
    return redirect(url_for("dashboard"))
@app.get("/menu")
def menu():
```

```
    sql = """
    SELECT
      mc.CATEGORY_NAME,
      mi.ITEM_ID,
      mi.NAME AS ITEM_NAME,
      mi.PRICE,
      mi.CALORIE_COUNT,
      mi.AVERAGE_RATING
    FROM MENU_ITEM mi
    JOIN MENU_CATEGORY mc ON mc.CATEGORY_ID = mi.CATEGORY_ID
    WHERE mi.IS_AVAILABLE = 'Y'
    ORDER BY mc.CATEGORY_NAME, mi.NAME
    """
    with POOL.acquire() as conn:
        with conn.cursor() as cur:
            cur.execute(sql)
            rows = cur.fetchall()

    grouped: dict[str, list[dict]] = {}
    for cat, item_id, name, price, cal, avg_rating in rows:
        grouped.setdefault(cat, []).append(
            {
                "item_id": int(item_id),
                "name": name,
                "price": float(price),
                "calorie_count": int(cal),
                "avg_rating": float(avg_rating),
            }
        )
    return render_template("menu.html", grouped=grouped)


@app.route("/order", methods=["GET", "POST"])
def order():
    with POOL.acquire() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT customer_id, name, customer_type FROM customer ORDER BY name"
            )
            customers = cur.fetchall()

            cur.execute(
                """
                SELECT mi.item_id, mc.category_name, mi.name, mi.price
                FROM menu_item mi
                JOIN menu_category mc ON mc.category_id = mi.category_id
                WHERE mi.is_available = 'Y'
                ORDER BY mc.category_name, mi.name
                """
            )
            items = cur.fetchall()

            cur.execute(
                "SELECT employee_id, name, role FROM employee WHERE role='Cashier' ORDER BY na
            )
            cashiers = cur.fetchall()

    if request.method == "GET":
        return render_template(
            "order.html",
            customers=customers,
            cashiers=cashiers,
            items=items,
        )
    customer_id = int(request.form.get("customer_id") or "0")
```

```
    employee_id = int(request.form.get("employee_id") or "0")
    payment_method = request.form.get("payment_method") or "Cash"

    chosen: list[str] = []
    for item_id, _, _, _ in items:
        qty_raw = request.form.get(f"qty_{item_id}", "").strip()
        if not qty_raw:
            continue
        try:
            qty = int(qty_raw)
        except ValueError:
            flash("Invalid quantity entered (must be a number).", "danger")
            return redirect(url_for("order"))
        if qty <= 0:
            continue
        chosen.append(f"{int(item_id)}:{qty}")

    if customer_id <= 0 or employee_id <= 0:
        flash("Please select a customer and a cashier.", "danger")
        return redirect(url_for("order"))
    if not chosen:
        flash("Please select at least one item with quantity.", "danger")
        return redirect(url_for("order"))

    try:
        with POOL.acquire() as conn:
            with conn.cursor() as cur:
                arr = cur.arrayvar(oracledb.DB_TYPE_VARCHAR, chosen)
                cur.callproc("CafeteriaManager.PlaceOrder", [customer_id, employee_id, arr])

                # Get the newly created order id for this session/customer (same txn)
                cur.execute(
                    """
                    SELECT order_id, total_amount
                    FROM "ORDER"
                    WHERE customer_id = :cid
                      AND order_date = TRUNC(SYSDATE)
                    ORDER BY order_id DESC
                    FETCH FIRST 1 ROWS ONLY
                    """,
                    {"cid": customer_id},
                )
                row = cur.fetchone()
                if not row:
                    raise RuntimeError("Order was placed but could not be located.")
                order_id, total_amount = int(row[0]), float(row[1])

                # Auto-process full payment for demo usability
                cur.callproc(
                    "CafeteriaManager.ProcessPayment",
                    [order_id, total_amount, payment_method],
                )
            conn.commit()

        flash(f"Order #{order_id} placed and paid successfully.", "success")
        return redirect(url_for("orders"))
    except Exception as ex:
        flash(f"Failed to place order: {ex}", "danger")
        return redirect(url_for("order"))

... (truncated) ...
```